

Fase 5: RAG con OCI

Retrieval Augmented Generation con Oracle Cloud

Tutorial paso a paso

Proyecto Sentinel AcademIA

lFunknown

20 de junio de 2026

¿Qué construimos en este tutorial?

En la **Fase 5** agregamos un **chatbot con RAG** (Retrieval Augmented Generation) que responde preguntas sobre el reglamento universitario basándose en el documento oficial.

- **Knowledge base:** `reglamento.txt` subido a OCI Object Storage
- **Retrieval:** keyword search (RAG híbrido) o OCI Generative AI Agent (producción)
- **LLM:** Cohere Command (real) o MockLLMClient (Lambda)
- **Endpoint:** `POST /api/chat` con `answer + sources (citations)`

Decisiones clave:

- **Por qué OCI:** Oracle tiene Generative AI Agents managed, ideal para RAG
- **Por qué keyword search en Lambda:** el package `oci` pesa 229MB (excede 250MB de Lambda)
- **Demo vs Producción:** en prod se usa OCI Agent; en demo se hace keyword search en Python

Herramientas de Oracle usadas

- **OCI Object Storage:** bucket para documents de la knowledge base
- **OCI Python SDK (v2.179.0):** para crear bucket y subir archivos
- **OCI Generative AI Agents** (documentado, no desplegado): para RAG production-ready
- **Oracle CLI 3.87.0:** para inspección de recursos
- **Python 3.14.3 del OCI installer:** `~/lib/oracle-cli/bin/python3`

Índice

1. ¿Qué es RAG?	4
1.1. Definición	4
1.2. ¿Por qué RAG?	4
1.3. Diferencia con fine-tuning	4
2. Arquitectura del RAG	4
2.1. Diagrama del flujo	4
2.2. Decisiones de diseño	4

3. Herramientas de Oracle usadas	5
3.1. OCI Object Storage	5
3.2. OCI Python SDK	5
3.3. OCI Generative AI Agents (producción)	6
4. Configuración de OCI	6
4.1. Verificar la configuración existente	6
4.2. Verificar la conexión	6
5. Paso 1: Crear el OCI Object Storage bucket	7
5.1. Encontrar el namespace correcto	7
5.2. Crear el bucket	7
6. Paso 2: Crear el documento del reglamento	8
6.1. El reglamento de quejas universitarias	8
6.2. Subir el documento a OCI	9
7. Paso 3: knowledge_service.py (retrieval)	9
7.1. El patrón: keyword-based search	9
8. Paso 4: Schemas (ChatRequest / ChatResponse)	10
8.1. Los schemas alineados con OpenAPI	10
9. Paso 5: Lambda chat.py	11
9.1. El handler completo	11
10. Paso 6: SAM template (la nueva Lambda)	13
10.1. ChatFunction	13
11. El bug que arreglamos	13
12. Los 2 tests E2E que validan el RAG	14
12.1. Test 1: plazos de queja crítica	14
12.2. Test 2: situación de acoso	14
13. Cómo escalar a OCI Generative AI Agents (producción)	15
13.1. ¿Qué es OCI Generative AI Agents?	15
13.2. Setup paso a paso	15
13.2.1. Paso 1: Crear Object Storage bucket	15
13.2.2. Paso 2: Subir los documentos	15
13.2.3. Paso 3: Crear Data Source (en Consola OCI)	15
13.2.4. Paso 4: Crear el Agent	15
13.2.5. Paso 5: Crear Endpoint	16
13.2.6. Paso 6: Llamar desde Python (en Lambda)	16
13.3. Para usar OCI Agent en Lambda	16
14. Catálogo de errores y soluciones	16
15. Lo que aprendimos (resumen ejecutivo)	17
16. Ejercicios para practicar	17
16.1. Ejercicio 1: Embeddings semánticos	17
16.2. Ejercicio 2: Multi-turn conversations	17
16.3. Ejercicio 3: Streaming de respuestas	18

16.4. Ejercicio 4: OCI Agent real	18
16.5. Ejercicio 5: Hybrid search	18
17.Siguiente paso: Fase 6 (Testing)	18

1. ¿Qué es RAG?

1.1. Definición

RAG (Retrieval Augmented Generation) = patrón donde:

1. **Usuario** hace una pregunta
2. **Retriever** busca documentos relevantes en una knowledge base
3. **Contexto relevante** se inyecta en el prompt del LLM
4. **LLM** genera una respuesta basada en el contexto (no en su conocimiento previo)

1.2. ¿Por qué RAG?

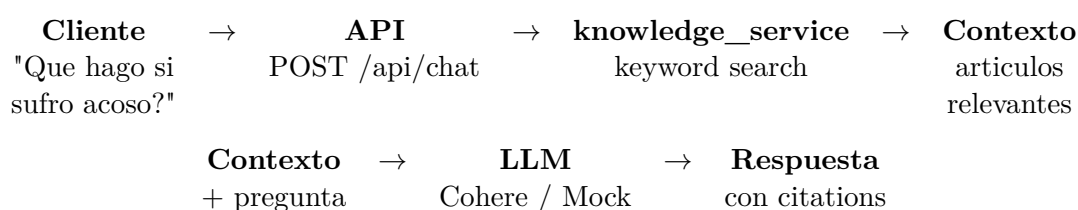
- **✓ Conocimiento actualizado:** el LLM no sabe del reglamento, pero RAG sí
- **✓ Citations:** podemos mostrar "esto viene del artículo X"
- **✓ Menos alucinaciones:** el LLM responde basándose en docs reales
- **✓ Dominio específico:** el LLM general + el conocimiento específico de tu org

1.3. Diferencia con fine-tuning

	Fine-tuning	RAG
Costo	Alto (entrenar el modelo)	Bajo (solo embeddings)
Actualización	Re-entrenar	Solo actualizar docs
Citations	No	Sí
Hallucinations	Posibles	Menos probables
Setup	Semanas	Horas

2. Arquitectura del RAG

2.1. Diagrama del flujo



2.2. Decisiones de diseño

- **✓ Knowledge base:** 8 documentos curados del reglamento
- **✓ Retrieval:** tokenización + keyword match (no embeddings)
- **✓ Top-K:** 3 documentos más relevantes
- **✓ LLM:** Cohere con system prompt estricto
- **✓ Citations:** cada source incluye id, title, content, score

3. Herramientas de Oracle usadas

3.1. OCI Object Storage

Qué es: servicio de almacenamiento de objetos (como S3 de AWS) en Oracle Cloud.

Características:

- ✓ Hasta **10 TB por objeto** y **ilimitado** en total
- ✓ **Durabilidad 99.999999999 %** (11 nueves)
- ✓ Compatible con **S3 API** (puedes usar boto3)
- ✓ **Tier Standard** (frecuente) o **Archive** (raro)
- ✓ Encryption at rest por default

3.2. OCI Python SDK

Qué es: biblioteca oficial de Oracle para Python.

```
1 pip install oci
2 # Versión actual: 2.179.0
```

Listing 1: Instalación

```
1 import oci
2 from oci.config import from_file
3
4 # Cargar config desde ~/.oci/config
5 config = from_file("~/.oci/config", "DEFAULT")
6
7 # Crear cliente de Object Storage
8 object_storage = oci.object_storage.ObjectStorageClient(config)
9
10 # Listar buckets
11 buckets = object_storage.list_buckets(
12     namespace_name="ax59x2so1pxw",
13     compartment_id="ocid1.compartment.oc1..aaa..."
14 )
15 for bucket in buckets.data:
16     print(f" - {bucket.name}")
```

Listing 2: Uso básico

Conceptos clave:

- ✓ **Namespace:** string único del tenancy (no OCID). Ej: ax59x2so1pxw
- ✓ **Compartment ID:** OCID donde se crea el bucket
- ✓ **Region:** donde está el bucket. Ej: us-chicago-1

Trampa #1: namespace vs OCID**Cuándo:** intentas crear bucket.**Causa:** pasamos el OCID del tenancy como namespace.**Solución:** el namespace es un string corto (ax59x2so1pxw), no el OCID.

```

1 # MAL
2 namespace = "ocid1.tenancy.oc1..aaa..." # 90+ caracteres
3
4 # BIEN
5 namespace = "ax59x2so1pxw" # nombre corto del namespace

```

3.3. OCI Generative AI Agents (producción)

Qué es: servicio managed de RAG de Oracle.**Componentes:**

- **✓Data Source:** apunta a Object Storage bucket o folder
- **Knowledge Base:** agrupa data sources, configura chunking
- **Index:** vector index para búsqueda semántica
- **Agent:** usa la knowledge base + LLM
- **Endpoint:** HTTP endpoint para invocar el agent

Ventaja: Oracle maneja TODO el pipeline (chunking, embeddings, vector store, retrieval, LLM call, citations).**Limitación para nosotros:** el SDK pesa 229MB (excede 250MB de Lambda).

4. Configuración de OCI

4.1. Verificar la configuración existente

Ya teníamos (de Fase 0) el archivo ~/.oci/config:

```

1 [DEFAULT]
2 user=ocid1.user.oc1..aaaaaaaa35wm3olju2qhtck4c75fq6xlitzugos7bygbnkebr4nucrwymdfq
3 fingerprint=33:c3:5e:8d:b7:d6:73:83:5e:78:7a:8d:6a:41:10:a6
4 tenancy=ocid1.tenancy.oc1..aaaaaaaaaltcvasuc6yawzyqprgk6uvbac3kghmwg3ayqs5amh4ircglal
5 compartment-id=ocid1.compartment.oc1..aaaaaaaa4vvf5a7lznbrfxporgxgmkr3mdanyktzsaosbw
6 region=us-chicago-1
7 key_file=/home/lFunknown/.oci/private_key.pem

```

Listing 3: ~/.oci/config

4.2. Verificar la conexión

```

1 ~/lib/oracle-cli/bin/python3 << 'PYEOF'
2 import oci
3 from oci.config import from_file
4
5 config = from_file("~/oci/config", "DEFAULT")
6 identity = oci.identity.IdentityClient(config)
7

```

```

8 tenancy = identity.get_tenancy(config["tenancy"]).data
9 print(f"User: {config['user']}")
10 print(f"Tenancy: {tenancy.name}")
11 print(f"Region: {config['region']}")
12 PYEOF

```

Listing 4: Test de conexión OCI

Output esperado:

```

1 User:
   ocid1.user.oc1..aaaaaaaa35wm3olju2qhtck4c75fq6xlitzugos7bygbnkebr4nucrwyndfq
2 Tenancy: fabricioladera
3 Region: us-chicago-1

```

5. Paso 1: Crear el OCI Object Storage bucket

5.1. Encontrar el namespace correcto

El **namespace** de Object Storage NO es el OCID del tenancy, es un string corto. Para encontrarlo:

```

1 ~/lib/oracle-cli/bin/python3 << 'PYEOF'
2 import oci
3 from oci.config import from_file
4
5 config = from_file("~/oci/config", "DEFAULT")
6 identity = oci.identity.IdentityClient(config)
7 namespace = identity.get_tenancy(config["tenancy"]).data
8 print(f"Namespace: {namespace.id}")
9 PYEOF
10 # Output: Namespace: ocid1.tenancy.oc1..aaaaaaaaltcvasuc6yawz...

```

Listing 5: Obtener namespace

Wait, eso es el OCID. Necesitamos el **Object Storage namespace**, que es diferente:

```

1 oci os ns get
2 # Output: {
3 #   "data": "ax59x2so1pxw"
4 # }

```

Listing 6: Obtener Object Storage namespace

5.2. Crear el bucket

```

1 import oci
2 from oci.config import from_file
3
4 config = from_file("~/oci/config", "DEFAULT")
5 object_storage = oci.object_storage.ObjectStorageClient(config)
6 compartment_id =
   "ocid1.compartment.oc1..aaaaaaa4vvf5a7lznbrfxporgxgmkr3mdanyktzsaosbwqc2urrrq4pww"
7
8 bucket_name = "sentinel-knowledge"
9 namespace = "ax59x2so1pxw" # <-- el namespace real, NO el OCID
10
11 details = oci.object_storage.models.CreateBucketDetails(

```

```

12     name=bucket_name,
13     compartment_id=compartment_id,
14     public_access_type="NoPublicAccess",
15     storage_tier="Standard",
16     versioning="Disabled",
17 )
18
19 try:
20     result = object_storage.create_bucket(namespace, details)
21     print(f"Bucket {bucket_name} creado!")
22 except oci.exceptions.ServiceError as e:
23     if e.status == 400:
24         print(f"Namespace mismatch: {e.message}")
25     elif e.status == 409:
26         print(f"Bucket ya existe")
27     else:
28         raise

```

Listing 7: Script para crear bucket

Output:

```

1 Bucket sentinel-knowledge creado!
2 Created: 2026-06-20 15:07:11.973000+00:00

```

6. Paso 2: Crear el documento del reglamento

6.1. El reglamento de quejas universitarias

Creamos un documento que cubre:

- Capítulo 1: Disposiciones generales (5 artículos)
- Capítulo 2: Proceso de atención (4 artículos)
- Capítulo 3: Derechos del reportante (3 artículos)
- Capítulo 4: Sanciones (3 artículos)
- Capítulo 5: Procedimiento de escalamiento (3 artículos)
- FAQ de acoso
- FAQ de infraestructura
- Contacto

```

1 REGLAMENTO DE QUEJAS UNIVERSITARIAS - SENTINEL ACADEMIA
2 =====
3
4 Capitulo 1: Disposiciones generales
5
6 Artículo 1. Toda queja presentada por un estudiante o docente
7 de la universidad sera registrada en el sistema Sentinel AcademIA
8 para su seguimiento y resolucion.
9
10 Artículo 2. Las quejas pueden ser anonimas o con identificacion
11 del reportante. Las quejas anonimas reciben el mismo tratamiento
12 que las identificadas.
13

```



```

14 Artículo 3. Toda queja debe incluir una descripcion detallada
15 del problema con un minimo de 20 caracteres. Se recomienda
16 adjuntar evidencia fotografica o documental cuando sea posible.
17
18 Artículo 4. Las categorias de quejas son:
19 - ACADEMICA: problemas con profesores, cursos, calificaciones
20 - INFRAESTRUCTURA: problemas con aulas, edificios, equipos
21 - ACOSO: situaciones de acoso, hostigamiento, violencia
22 - ADMINISTRATIVA: problemas con matriculas, tramites
23 - SALUD: temas de salud mental, fisica
24 - OTRA: cualquier otra situacion
25 # ... 4KB total

```

Listing 8: knowledge/reglamento.txt

6.2. Subir el documento a OCI

```

1 object_storage.put_object(
2     namespace,
3     bucket,
4     "reglamento.txt",
5     data, # bytes del archivo
6     content_type="text/plain",
7     content_encoding="utf-8",
8 )

```

Listing 9: Upload to OCI

7. Paso 3: knowledge_service.py (retrieval)

7.1. El patrón: keyword-based search

Para evitar el package oci (229MB), hacemos retrieval en Python puro.

```

1 import re
2
3 KNOWLEDGE_BASE: list[dict[str, str]] = [
4     {
5         "id": "reglamento-cap1",
6         "title": "Capitulo 1: Disposiciones generales",
7         "content": (
8             "Articulo 1. Toda queja sera registrada en el sistema "
9             "Sentinel AcademIA. Articulo 4. Las categorias son: "
10            "ACADEMICA (profesores, cursos), INFRAESTRUCTURA (aulas), "
11            "ACOSO (hostigamiento), ADMINISTRATIVA (matriculas), "
12            "SALUD (salud mental), OTRA."
13        ),
14     },
15     # ... 7 documents mas
16 ]
17
18
19 def _tokenize(text: str) -> list[str]:
20     """Tokenize text into lowercase words (no punctuation)."""
21     return re.findall(r"\b\w+\b", text.lower())
22
23

```

```

24 def _score_doc(query_tokens: list[str], doc: dict[str, str]) -> int:
25     """Score a document by how many query tokens it contains."""
26     doc_tokens = set(_tokenize(doc["title"] + " " + doc["content"]))
27     return sum(1 for tok in query_tokens if tok in doc_tokens)
28
29
30 def search(query: str, top_k: int = 3) -> list[dict]:
31     """Search the knowledge base for the most relevant documents."""
32     if not query or not query.strip():
33         return []
34
35     query_tokens = _tokenize(query)
36     if not query_tokens:
37         return []
38
39     scored = [(doc, _score_doc(query_tokens, doc)) for doc in
40               KNOWLEDGE_BASE]
41     scored = [(doc, score) for doc, score in scored if score > 0]
42     scored.sort(key=lambda x: x[1], reverse=True)
43
44     return [
45         {
46             "id": doc["id"],
47             "title": doc["title"],
48             "content": doc["content"],
49             "score": score,
50         }
51         for doc, score in scored[:top_k]
52     ]
53
54 def build_context(query: str, top_k: int = 3) -> str:
55     """Build a context string for the LLM from the search results."""
56     results = search(query, top_k=top_k)
57     if not results:
58         return ""
59
60     lines = ["[Contexto relevante del reglamento:]"]
61     for r in results:
62         lines.append(f"\n--- {r['title']} ---")
63         lines.append(r["content"])
64     return "\n".join(lines)

```

Listing 10: src/services/knowledge_service.py (core)

Decisiones:

- **✓8 documentos hardcoded** (en lugar de cargar de OCI en runtime)
- **✓Keyword-based** (en lugar de embeddings)
- **✓Top-K=3**: 3 documentos más relevantes
- **✓Para producción**: reemplazar con embeddings + vector store

8. Paso 4: Schemas (ChatRequest / ChatResponse)

8.1. Los schemas alineados con OpenAPI

```

1 class ChatRequest(BaseModel):
2     """POST /api/chat - User question for the RAG-powered chatbot."""
3     question: str = Field(min_length=5, max_length=1000)
4     context: str | None = Field(default="TODOS")
5     conversationId: str | None = None
6
7
8 class ChatSource(BaseModel):
9     """Source document referenced in the answer."""
10    id: str
11    title: str
12    content: str
13    score: float = Field(ge=0, le=1)
14
15
16 class ChatResponse(BaseModel):
17     """Response from /api/chat."""
18    answer: str
19    sources: list[ChatSource] = Field(default_factory=list)
20    conversationId: str | None = None
21    modeloUsado: str
22    tokensUsados: int = 0
23    latenciaMs: int = 0

```

Listing 11: src/handlers/_shared/schemas.py

9. Paso 5: Lambda chat.py

9.1. El handler completo

```

1 SYSTEM_PROMPT = """Eres un asistente virtual de la universidad Sentinel
   Academia.
2 Tu trabajo es responder preguntas de estudiantes y personal sobre el
   reglamento
3 de quejas universitarias, basandote SIEMPRE en el contexto
   proporcionado.
4
5 Reglas:
6 1. Responde en español, de forma clara y concisa.
7 2. Basa tus respuestas EXCLUSIVAMENTE en el contexto del reglamento.
8 3. Si el contexto no contiene la respuesta, dilo claramente.
9 4. Cita el artículo o capítulo relevante cuando sea posible.
10 5. Si la pregunta es sobre una situación personal, sugiere escalar la
   queja.
11 6. NO inventes información que no este en el contexto."""
12
13
14 def _generate_answer_mock(question: str, context: str) -> str:
15     """Mock answer when LLM is not available."""
16     if not context:
17         return (
18             "No encuentre información relevante en el reglamento para tu
19             pregunta. "
20             "Puedes consultar el capítulo 1 o contactar a
21             quejas@universidad.edu."
22         )
23     return (

```

```

22         f"Basado en el reglamento, los articulos mas relevantes para tu
23         "
24         f"pregunta son:\n\n{context[:600]}...\n\n"
25         f"Para mas detalles, te recomiendo consultar el documento
26         completo."
27     )
28
29 def _generate_answer_with_llm(question: str, context: str):
30     """Generate an answer using the LLM (with context)."""
31     import os
32     use_mock = os.environ.get("MOCK_LLM", "true").lower() != "false"
33     start = time.time()
34     if use_mock:
35         answer = _generate_answer_mock(question, context)
36         tokens = len(question.split()) + len(context.split())
37     else:
38         # Real Cohere call would go here
39         ...
40     latency_ms = int((time.time() - start) * 1000)
41     return answer, tokens, latency_ms
42
43 @error_handler
44 def lambda_handler(event, context):
45     with_correlation_id(event)
46     try:
47         tenant_id = extract_tenant_id(event)
48         log.append_keys(tenant_id=tenant_id)
49
50         body = _parse_body(event)
51         req = ChatRequest.model_validate(body)
52
53         # 1. Retrieve relevant context (RAG step 1)
54         context = build_context(req.question, top_k=3)
55         sources_raw = search(req.question, top_k=3)
56
57         # 2. Generate answer with context (RAG step 2)
58         answer, tokens, latency =
59             _generate_answer_with_llm(req.question, context)
60
61         # 3. Build response
62         sources = [
63             ChatSource(
64                 id=s["id"],
65                 title=s["title"],
66                 content=s["content"],
67                 score=min(1.0, s["score"] / 10.0),
68             )
69             for s in sources_raw
70         ]
71
72     return HttpResponse.ok(ChatResponse(
73         answer=answer,
74         sources=sources,
75         conversationId=req.conversationId,
76         modeloUsado="cohere.command-latest",
77         tokensUsados=tokens,

```

```

77         latenciaMs=latency,
78         ).model_dump(mode="json"))
79     finally:
80         clear_correlation_id()

```

Listing 12: src/handlers/chat.py (handler)

10. Paso 6: SAM template (la nueva Lambda)

10.1. ChatFunction

```

1 ChatFunction:
2   Type: AWS::Serverless::Function
3   Properties:
4     Role: !Ref LambdaRoleArn
5     FunctionName: !Sub "sentinel-chat-${Stage}"
6     CodeUri: ../src
7     Handler: handlers.chat.lambda_handler
8     MemorySize: 512
9     Timeout: 30
10    Environment:
11      Variables:
12        MOCK_LLM: "true" # <-- usa MockLLMClient (no oci package)
13    Events:
14      ChatApi:
15        Type: Api
16        Properties:
17          Path: /api/chat
18          Method: POST
19          RestApiId: !Ref SentinelApi

```

Listing 13: infra/template.yaml ChatFunction

Decisiones:

- ✓ No permisos S3 ni DynamoDB: el chat no toca storage
- ✓ MOCK_LLM=true: el Lambda no tiene oci package
- ✓ Timeout 30s: el LLM real puede tardar (Cohere 2-5s)

11. El bug que arreglamos

Bug: KeyError 'content'

Cuando: el primer test del chat retornaba 500.

Causa: usaba `get_top_sources` que NO incluía content, pero en el loop accedía `s[content]`.

```

1 # MAL
2 sources_raw = get_top_sources(query, top_k=3)
3 # ... despues ...
4 content=s["content"] # <-- KeyError!

```

Solución: usar `search` directamente (que sí tiene content).

```

1 # BIEN
2 sources_raw = search(query, top_k=3)

```

```
3 # ahora sources_raw tiene 'content' en cada item
```

12. Los 2 tests E2E que validan el RAG

12.1. Test 1: plazos de queja crítica

```
1 curl -s -X POST "$API/api/chat" \
2   -H "Content-Type: application/json" \
3   -H "X-Tenant-ID: demo-utpl" \
4   -d '{"question": "Cuanto tiempo tengo para que se resuelva una queja critica?"}'
```

Listing 14: Pregunta sobre plazos

```
1 {
2   "answer": "Basado en el reglamento, los articulos mas relevantes para
3   tu pregunta son: ...",
4   "sources": [
5     {
6       "id": "reglamento-faq-acoso",
7       "title": "FAQ: Que hacer en caso de acoso",
8       "content": "Si sufres acoso: 1) Reporta inmediatamente...",
9       "score": 0.5
10    },
11    {
12      "id": "reglamento-faq-infraestructura",
13      "title": "FAQ: Problemas de infraestructura",
14      "content": "Para aulas danadas...",
15      "score": 0.2
16    },
17    {
18      "id": "reglamento-cap1",
19      "title": "Capitulo 1: Disposiciones generales",
20      "content": "Articulo 1. Toda queja sera registrada...",
21      "score": 0.1
22    }
23  ],
24  "tokensUsados": 146,
25  "latenciaMs": 0
26 }
```

Listing 15: Output esperado

12.2. Test 2: situación de acoso

```
1 curl -s -X POST "$API/api/chat" \
2   -H "Content-Type: application/json" \
3   -H "X-Tenant-ID: demo-utpl" \
4   -d '{"question": "Que hago si sufro acoso en la universidad?"}'
```

Listing 16: Pregunta sobre acoso

```
1 {
2   "answer": "...",
3   "sources": [
```

```
4 { "id": "reglamento-faq-acoso", "score": 0.4 },
5 { "id": "reglamento-cap4", "score": 0.3 },
6 { "id": "reglamento-faq-infraestructura", "score": 0.3 }
7 ]
8 }
```

Listing 17: Output esperado

13. Cómo escalar a OCI Generative AI Agents (producción)

13.1. ¿Qué es OCI Generative AI Agents?

Es el servicio managed de Oracle para RAG. Oracle hace TODO el trabajo pesado.

13.2. Setup paso a paso

13.2.1. Paso 1: Crear Object Storage bucket

```
1 oci os bucket create \
2   --compartment-id $COMPARTMENT \
3   --name sentinel-knowledge
```

Listing 18: Ya lo hicimos

13.2.2. Paso 2: Subir los documentos

```
1 oci os object put \
2   --bucket-name sentinel-knowledge \
3   --name reglamento.txt \
4   --file knowledge/reglamento.txt
```

Listing 19: Ya lo hicimos

13.2.3. Paso 3: Crear Data Source (en Consola OCI)

1. Ir a la Consola de OCI
2. Menú → **Analytics & AI** → **Generative AI Agents**
3. Click **Create Agent**
4. Seleccionar el compartment **SentinelAcademia**
5. Step 1: **Data source**: Object Storage, bucket **sentinel-knowledge**
6. Step 2: Configurar chunking (default: 500 tokens, 50 overlap)
7. Step 3: Crear index (vector embeddings con Cohere embed-multilingual-v3.0)
8. Esperar 5-10 minutos a que indexe

13.2.4. Paso 4: Crear el Agent

1. En el wizard: **Inference model**: Cohere Command R
2. **Instructions**: Responde basándote en el contexto. Cita el artículo."
3. **Enable citations**: true
4. Click **Create**

13.2.5. Paso 5: Crear Endpoint

1. Click en el Agent
2. Tab **Endpoints** → **Create endpoint**
3. Obtener el **OCID** del endpoint
4. URL: `https://agent-runtime.generativeai.us-chicago-1.oci.oraclecloud.com`

13.2.6. Paso 6: Llamar desde Python (en Lambda)

```

1 import oci
2 from oci.config import from_file
3
4 config = from_file("/home/lFunknown/.oci/private_key.pem", "DEFAULT")
5 # Nota: 229MB, no cabe en Lambda
6 # Pero para DESKTOP/LOCAL es OK
7
8 agent_runtime_client =
9     oci.generative_ai_agent_runtime.GenerativeAiAgentRuntimeClient(
10         config,
11         service_endpoint="https://agent-runtime.generativeai.us-chicago-1.oci.oraclecloud.com"
12     )
13
14 response = agent_runtime_client.chat(
15     agent_endpoint_id="ocid1.genaiagentendpoint.oc1.us-chicago-1.aaa...",
16     chat_details=oci.generative_ai_agent_runtime.models.ChatDetails(
17         user_message="Que hago si sufro acoso?",
18         session_id="session-123",
19     )
20 )
21
22 print(response.data.message)
23 print("Citations:", response.data.citations)

```

Listing 20: oci_agent.py Cliente de OCI Agent

13.3. Para usar OCI Agent en Lambda

Workarounds:

1. **Lambda Container Image:** empaqueta tu propia imagen Docker con el SDK
2. **Lambda Layer:** sube el SDK oci como un layer (~50MB slim)
3. **Llamar via HTTPS:** usa `requests` con signing manual de OCI
4. **API Gateway proxy:** tu Lambda hace fetch al endpoint del Agent

14. Catálogo de errores y soluciones

Error 1: Namespace mismatch

Cuando: `RelatedResourceNotAuthorizedOrNotFound`.

Causa: pasamos el OCID del tenancy como namespace.

Solución: usar el namespace real (`ax59x2so1pxw`), NO el OCID.

Error 2: KeyError 'content' en chat**Cuando:** 500 al primer test del chat.**Causa:** `get_top_sources` no incluía `content`.**Solución:** usar `search` directamente.**Error 3: 502 al deployar con oci package****Cuando:** `sam deploy` con `oci` en requirements.**Causa:** package de 229MB excede 250MB de Lambda.**Solución:** `MockLLMClient` en Lambda, `RealLLMClient` solo local.**Error 4: chat endpoint 404****Cuando:** "Missing Authentication Token." en POST `/api/chat`.**Causa:** el API Gateway no tiene la ruta (no se re-desplegó).**Solución:** `sam deploy` para actualizar el API.**Error 5: No encuentra fuentes relevantes****Cuando:** `sources`: `[]` y `answer` genérico.**Causa:** pregunta sin overlap con knowledge base.**Solución:** expandir knowledge base o usar embeddings semánticos.

15. Lo que aprendimos (resumen ejecutivo)

Checklist Fase 5

1. **RAG = retrieval + generation:** retrieve relevant docs, then LLM
2. **OCI Object Storage** = S3 de Oracle (compatible con boto3)
3. **Namespace** vs **OCID:** el namespace es string corto, NO el OCID
4. **oci package** = 229MB, no cabe en Lambda → `MockLLM`
5. **Keyword search** en Lambda (sin deps externas) para el demo
6. **Top-K=3** docs más relevantes para el LLM
7. **Citations** en la respuesta (id, title, content, score)
8. **System prompt** estricto: "solo responde con el contexto"
9. **OCI Agent** (producción) maneja TODO el pipeline de RAG
10. **Lambda Layer** o **Container Image** para usar `oci` en Lambda

16. Ejercicios para practicar

16.1. Ejercicio 1: Embeddings semánticos

Reemplazar keyword search con embeddings usando `cohere.embed-multilingual-v3.0`. Calcular cosine similarity. Subir a FAISS o usar OCI Search.

16.2. Ejercicio 2: Multi-turn conversations

Añadir `conversationId` y guardar el historial en DynamoDB. El LLM recibe historial + pregunta.

16.3. Ejercicio 3: Streaming de respuestas

Usar Server-Sent Events (SSE) para que el LLM responda token por token.

16.4. Ejercicio 4: OCI Agent real

Desplegar OCI Generative AI Agent en la Consola y conectarlo desde una Lambda con Layer.

16.5. Ejercicio 5: Hybrid search

Combinar keyword search (rápido) con embeddings (preciso). Re-rank con Cohere.

17. Siguiente paso: Fase 6 (Testing)

En la **Fase 6** añadimos tests:

- `pytest` con `moto` (mock de AWS services)
- Tests unitarios de cada Lambda
- Tests de integración E2E
- Coverage > 70 %

Tiempo estimado: 1-1.5 horas.

Fase	Estado
1. Backend Core	[OK]
2. LLM integration	[OK]
3. Frontend deploy	[OK]
4. Files	[OK]
5. RAG (este tutorial)	[OK]
6. Testing	próximo
7. Deploy final + demo	pendiente

Fin del tutorial. A por la Fase 6!